

Théorie des Réseaux de Neurones

Auteur : Manus AI

Introduction progressive

Pourquoi les réseaux de neurones ?

Les modèles linéaires, bien que simples et interprétables, rencontrent des limites fondamentales lorsqu'il s'agit de modéliser des relations complexes et non-linéaires dans les données. Par exemple, une régression linéaire ne peut pas capturer des motifs comme la relation XOR (ou exclusif), qui est non-linéaire et non-séparable linéairement. Les réseaux de neurones ont émergé comme une solution puissante pour surmonter ces limitations en introduisant des couches de transformations non-linéaires, permettant ainsi d'apprendre des représentations de données de plus en plus abstraites et complexes.

Fondements mathématiques cachés

Modèle du neurone formel (McCulloch & Pitts)

Le neurone formel, proposé par McCulloch et Pitts en 1943, est le bloc de construction fondamental des réseaux de neurones. Il s'agit d'un modèle simplifié du neurone biologique, capable de recevoir des entrées, de les pondérer, de les sommer, puis d'appliquer une fonction d'activation pour produire une sortie. Mathématiquement, pour un neurone donné, la sortie y est calculée comme suit :

$$y = f \left(\sum_{i=1}^n w_i x_i + b \right)$$

Où :

- x_i sont les entrées
- w_i sont les poids associés à chaque entrée
- b est le biais
- f est la fonction d'activation

Représentation vectorielle : $y = f(W \cdot x + b)$

Cette formule peut être exprimée de manière plus compacte en utilisant la notation vectorielle, ce qui est particulièrement utile pour les calculs dans les réseaux de neurones. Si $\mathbf{x}$ est le vecteur des entrées et $\mathbf{w}$ est le vecteur des poids, alors la somme pondérée des entrées est un produit scalaire :

$$y = f(\mathbf{w}^T \mathbf{x} + b)$$

Pour une couche entière de neurones, où $\mathbf{W}$ est la matrice des poids et $\mathbf{x}$ est le vecteur d'entrée, la sortie $\mathbf{z}$ avant l'activation est :

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

Ensuite, la sortie activée de la couche est $\mathbf{a} = f(\mathbf{z})$.

Le **produit scalaire** (ou produit interne) $\mathbf{w}^T \mathbf{x}$ peut être interprété comme une mesure de similarité entre les vecteurs $\mathbf{w}$ et $\mathbf{x}$. Géométriquement, il représente la projection du vecteur $\mathbf{x}$ sur le vecteur $\mathbf{w}$ (multipliée par la magnitude de $\mathbf{w}$). La **séparabilité linéaire** est la capacité d'un modèle à diviser les points de données en différentes classes à l'aide d'une seule ligne (en 2D), d'un plan (en 3D) ou d'un hyperplan (en dimensions supérieures).

Pourquoi le biais (b) est indispensable

Le terme de biais b est un paramètre crucial qui permet au neurone de décaler la fonction d'activation. Sans biais, la fonction d'activation passerait toujours par l'origine (0,0). Cela limiterait considérablement la capacité du modèle à apprendre des motifs complexes, car il ne pourrait pas modéliser des relations où la sortie est non-nulle lorsque toutes les entrées sont nulles. Le biais offre une flexibilité supplémentaire, permettant au modèle de mieux s'adapter aux données en ajustant l'interception de l'hyperplan de décision.

Perceptron

Définition, algorithme d'apprentissage

Le **Perceptron** est l'un des premiers algorithmes d'apprentissage supervisé pour la classification binaire, inventé par Frank Rosenblatt en 1957. Il s'agit d'un modèle linéaire qui prend plusieurs entrées, les pondère, les somme, ajoute un biais, puis applique une fonction d'activation (généralement une fonction signe) pour produire une sortie binaire (par exemple, -1 ou 1, ou 0 ou 1).

L'algorithme d'apprentissage du Perceptron est itératif et ajuste les poids et le biais à chaque erreur de classification. Si la prédiction est incorrecte, les poids sont mis à jour selon la règle suivante :

$$w_i \leftarrow w_i + \alpha(y - \hat{y})x_i$$

$$b \leftarrow b + \alpha(y - \hat{y})$$

Où :

- α est le taux d'apprentissage
- y est la vraie étiquette
- $\hat{y}$ est la prédiction du perceptron

Limite du perceptron simple (XOR) → besoin de couches cachées

Le perceptron simple est limité aux problèmes de classification linéairement séparables. Cela signifie qu'il ne peut pas résoudre des problèmes où les classes ne peuvent pas être séparées par une seule ligne droite (ou hyperplan). Le problème du XOR (ou exclusif) est un exemple classique de cette limitation. Pour résoudre des problèmes non-linéaires comme le XOR, il est nécessaire d'introduire des **couches cachées** et des fonctions d'activation non-linéaires, ce qui conduit à la notion de réseaux de neurones multi-couches (ou *Multi-Layer Perceptrons - MLP*).

Fonctions d'activation

Les fonctions d'activation introduisent la non-linéarité dans les réseaux de neurones, permettant ainsi au modèle d'apprendre des relations complexes. Sans elles, un réseau de neurones, même avec plusieurs couches, se comporterait comme un simple modèle linéaire.

Fonction d'activation	Formule	Dérivée	Rôle / Caractéristiques	Limites	Quand l'utiliser
Sigmoïde	$\sigma(x) = \frac{1}{1+e^{-x}}$	$\sigma'(x) = \sigma(x)(1 - \sigma(x))$	Comprime les valeurs entre 0 et 1. Souvent utilisée pour la classification binaire en couche de sortie.	Problème de <i>vanishing gradient</i> (gradient évanescent) pour les valeurs extrêmes. Sortie non centrée à zéro.	Couche de sortie pour classification binaire. Historiquement utilisée dans les couches cachées, mais moins maintenant.
Tanh (Tangente hyperbolique)	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	$\tanh'(x) = 1 - \tanh^2(x)$	Comprime les valeurs entre -1 et 1. Sortie centrée à zéro, ce qui aide à la convergence.	Problème de <i>vanishing gradient</i> pour les valeurs extrêmes.	Couches cachées, meilleure alternative à la sigmoïde.
ReLU (Rectified Linear Unit)	$\text{ReLU}(x) = \max(0, x)$	$\text{ReLU}'(x) = \begin{cases} 1 & \text{si } x > 0 \\ 0 & \text{si } x \leq 0 \end{cases}$	Simple et efficace. Aide à atténuer le problème de <i>vanishing gradient</i> pour les valeurs positives. Accélère la convergence.	Problème de <i>dying ReLU</i> : les neurones peuvent devenir inactifs si leur entrée est toujours négative.	Couches cachées dans la plupart des réseaux de neurones profonds.
Leaky ReLU	$\text{Leaky ReLU}(x) = \begin{cases} x & \text{si } x > 0 \\ \alpha x & \text{si } x \leq 0 \end{cases}$	$\text{Leaky ReLU}'(x) = \begin{cases} 1 & \text{si } x > 0 \\ \alpha & \text{si } x \leq 0 \end{cases}$	Une variante de ReLU qui tente de résoudre le problème de <i>dying ReLU</i> en permettant un petit gradient pour les entrées négatives (α est un petit nombre positif, par exemple 0.01).	Peut ne pas toujours surpasser ReLU.	Alternative à ReLU lorsque le problème de <i>dying ReLU</i> est observé.
Softmax	$\text{Softmax}(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}$	La dérivée est plus complexe et dépend de la	Transforme un vecteur de nombres réels en	Peut être sensible aux valeurs	Couche de sortie pour la

Fonction d'activation	Formule	Dérivée	Rôle / Caractéristiques	Limites	Quand l'utiliser
		fonction de coût (généralement l'entropie croisée).	une distribution de probabilité. La somme des sorties est égale à 1. Utilisée pour la classification multi-classes.	extrêmes dans les entrées.	classification multi-classes.

Architecture d'un réseau

Un réseau de neurones est généralement organisé en couches. Chaque couche est composée de plusieurs neurones qui effectuent des calculs similaires.

Couche d'entrée, couches cachées, couche de sortie

- **Couche d'entrée** : Reçoit les données brutes. Le nombre de neurones dans cette couche est égal au nombre de caractéristiques (features) dans les données d'entrée.
- **Couches cachées** : Ce sont les couches intermédiaires entre la couche d'entrée et la couche de sortie. Elles sont appelées «cachées» car leurs activations ne sont pas directement observées depuis l'extérieur du réseau. Elles apprennent des représentations de plus en plus abstraites des données.
- **Couche de sortie** : Produit le résultat final du réseau. Le nombre de neurones et la fonction d'activation de cette couche dépendent du type de tâche (par exemple, un neurone avec sigmoïde pour la classification binaire, plusieurs neurones avec softmax pour la classification multi-classes, ou un neurone linéaire pour la régression).

Notation L couches, $n[l]$ neurones

Pour décrire l'architecture d'un réseau de neurones, on utilise souvent la notation suivante :

- L : Nombre total de couches dans le réseau (en excluant la couche d'entrée).
- $n^{[l]}$: Nombre de neurones dans la l -ième couche.

Par exemple, un réseau avec une couche d'entrée de 784 neurones (pour des images MNIST de 28x28 pixels), une première couche cachée de 128 neurones, une deuxième couche cachée de 64 neurones et une couche de sortie de 10 neurones (pour 10 classes) aurait :

- Couche d'entrée : $n^{[0]} = 784$
- Couche cachée 1 : $n^{[1]} = 128$
- Couche cachée 2 : $n^{[2]} = 64$
- Couche de sortie : $n^{[3]} = 10$

- Nombre total de couches (hors entrée) : $L = 3$

Pourquoi plus de neurones / plus de couches ?

- **Plus de neurones par couche** : Augmenter le nombre de neurones dans une couche permet au réseau de capturer des motifs plus complexes et d'avoir une plus grande capacité de représentation. Cependant, cela augmente également le risque de sur-apprentissage et la complexité computationnelle.
- **Plus de couches (réseaux profonds)** : Les réseaux profonds (avec plusieurs couches cachées) peuvent apprendre des hiérarchies de caractéristiques. Les premières couches apprennent des caractéristiques de bas niveau (par exemple, les bords dans une image), tandis que les couches plus profondes combinent ces caractéristiques pour former des représentations de plus haut niveau (par exemple, des formes ou des objets). Cette architecture hiérarchique est essentielle pour résoudre des problèmes complexes en vision par ordinateur ou en traitement du langage naturel.

Rétropropagation du gradient (théorie complète)

La rétropropagation du gradient est l'algorithme fondamental qui permet aux réseaux de neurones d'apprendre. Elle consiste à calculer le gradient de la fonction de coût par rapport à chaque poids et biais du réseau, puis à ajuster ces paramètres dans la direction qui minimise la fonction de coût.

Fonction coût : erreur quadratique, entropie croisée

La **fonction de coût** (ou fonction de perte) mesure à quel point les prédictions du réseau sont éloignées des vraies étiquettes. L'objectif de l'apprentissage est de minimiser cette fonction.

- **Erreur quadratique moyenne (Mean Squared Error - MSE)** : Souvent utilisée pour les problèmes de régression.

$$J(\mathbf{w}, b) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$$

Où :

- m est le nombre d'exemples d'entraînement
- $\hat{y}^{(i)}$ est la prédiction du réseau pour l'exemple i
- $y^{(i)}$ est la vraie étiquette pour l'exemple i
- **Entropie croisée (Cross-Entropy)** : Couramment utilisée pour les problèmes de classification.

Pour la classification binaire (avec une fonction sigmoïde en sortie) :

$$J(\mathbf{w}, b) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$

Pour la classification multi-classes (avec une fonction softmax en sortie) :

$$J(\mathbf{w}, b) = -\frac{1}{m} \sum_{i=1}^m \sum_{j=1}^K y_j^{(i)} \log(\hat{y}_j^{(i)})$$

Où :

- K est le nombre de classes
- $y_j^{(i)}$ est 1 si l'exemple i appartient à la classe j , et 0 sinon
- $\hat{y}_j^{(i)}$ est la probabilité prédite par le réseau que l'exemple i appartienne à la classe j

Descente de gradient stochastique (SGD)

La **descente de gradient stochastique (SGD)** est un algorithme d'optimisation qui ajuste itérativement les poids et les biais du réseau pour minimiser la fonction de coût. Au lieu de calculer le gradient sur l'ensemble du jeu de données (ce qui peut être très coûteux), SGD calcule le gradient sur un petit sous-ensemble aléatoire des données, appelé **mini-batch**. La mise à jour des paramètres se fait selon la règle :

$$\begin{aligned}\mathbf{w} &\leftarrow \mathbf{w} - \alpha \frac{\partial J}{\partial \mathbf{w}} \\ b &\leftarrow b - \alpha \frac{\partial J}{\partial b}\end{aligned}$$

Où α est le taux d'apprentissage, qui contrôle la taille des pas de mise à jour.

Calcul des gradients par chaînage (dérivée composée)

La rétropropagation utilise la règle de la chaîne pour calculer les gradients de la fonction de coût par rapport à chaque poids et biais. L'idée est de calculer les gradients de la couche de sortie vers la couche d'entrée, en réutilisant les gradients calculés précédemment. Pour un réseau simple, le gradient d'une fonction composée $f(g(x))$ est $f'(g(x)) \cdot g'(x)$. En général, pour un réseau de neurones, cela implique de calculer comment une petite modification d'un poids ou d'un biais affecte la sortie du neurone, puis comment cette modification affecte la sortie de la couche suivante, et ainsi de suite jusqu'à la fonction de coût.

Détail mathématique pas à pas pour un réseau à 1 couche cachée

Considérons un réseau de neurones simple avec une couche d'entrée, une couche cachée et une couche de sortie. Supposons que nous utilisons la fonction d'activation sigmoïde pour la couche cachée et la couche de sortie, et l'erreur quadratique comme fonction de coût.

Phase avant (Forward Pass) :

1. Couche cachée :

- Entrée pondérée : $z^{[1]} = W^{[1]}x + b^{[1]}$

- Activation : $a^{[1]} = \sigma(z^{[1]})$

2. Couche de sortie :

- Entrée pondérée : $z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$
- Activation (prédiction) : $\hat{y} = a^{[2]} = \sigma(z^{[2]})$

Phase arrière (Backward Pass) :

L'objectif est de calculer $\frac{\partial J}{\partial W^{[2]}}$, $\frac{\partial J}{\partial b^{[2]}}$, $\frac{\partial J}{\partial W^{[1]}}$ et $\frac{\partial J}{\partial b^{[1]}}$.

1. Gradient de la fonction de coût par rapport à la sortie :

- Pour MSE : $\frac{\partial J}{\partial \hat{y}} = (\hat{y} - y)$

2. Gradient de la couche de sortie :

- $\frac{\partial J}{\partial z^{[2]}} = \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z^{[2]}} = (\hat{y} - y) \cdot \sigma'(z^{[2]})$
- $\frac{\partial J}{\partial W^{[2]}} = \frac{\partial J}{\partial z^{[2]}} \cdot (a^{[1]})^T$
- $\frac{\partial J}{\partial b^{[2]}} = \frac{\partial J}{\partial z^{[2]}}$

3. Gradient de la couche cachée :

- $\frac{\partial J}{\partial a^{[1]}} = (W^{[2]})^T \frac{\partial J}{\partial z^{[2]}}$
- $\frac{\partial J}{\partial z^{[1]}} = \frac{\partial J}{\partial a^{[1]}} \cdot \sigma'(z^{[1]})$
- $\frac{\partial J}{\partial W^{[1]}} = \frac{\partial J}{\partial z^{[1]}} \cdot x^T$
- $\frac{\partial J}{\partial b^{[1]}} = \frac{\partial J}{\partial z^{[1]}}$

Ces gradients sont ensuite utilisés pour mettre à jour les poids et les biais via la descente de gradient.

Initialisation des poids

Pourquoi ne pas initialiser à zéro ?

Initialiser tous les poids à zéro est une mauvaise pratique. Si tous les poids sont initialisés à zéro, alors chaque neurone dans une couche donnée effectuera exactement les mêmes calculs et aura les mêmes gradients lors de la rétropropagation. Cela signifie que tous les neurones d'une même couche apprendront les mêmes caractéristiques, ce qui rend le réseau incapable d'apprendre des représentations diverses et complexes. Le réseau perdrait sa capacité à apprendre des motifs différents et se comporterait comme un modèle linéaire.

Initialisation de Xavier / He

Pour éviter le problème de l'initialisation à zéro et pour aider à maintenir les activations des neurones dans une plage raisonnable (ni trop petites, ni trop grandes), des méthodes d'initialisation spécifiques ont été développées :

- **Initialisation de Xavier (Glorot) :** Recommandée pour les fonctions d'activation sigmoïde et tanh. Les poids sont tirés d'une distribution uniforme ou normale avec une variance qui dépend du nombre de neurones d'entrée et de sortie de la couche.

- Pour une distribution uniforme : $W \sim U(-\sqrt{\frac{6}{n_{in}+n_{out}}}, \sqrt{\frac{6}{n_{in}+n_{out}}})$

- Pour une distribution normale : $W \sim N(0, \sqrt{\frac{2}{n_{in}+n_{out}}})$

Où n_{in} est le nombre de neurones d'entrée et n_{out} est le nombre de neurones de sortie de la couche.

- **Initialisation de He :** Spécifiquement conçue pour les fonctions d'activation ReLU et ses variantes. Elle est similaire à Xavier mais prend en compte le fait que ReLU met à zéro la moitié des activations.

- Pour une distribution uniforme : $W \sim U(-\sqrt{\frac{6}{n_{in}}}, \sqrt{\frac{6}{n_{in}}})$

- Pour une distribution normale : $W \sim N(0, \sqrt{\frac{2}{n_{in}}})$

Où n_{in} est le nombre de neurones d'entrée de la couche.

Ces méthodes aident à prévenir l'évanouissement ou l'explosion des gradients en assurant que les activations et les gradients restent dans une plage saine tout au long de l'entraînement.

Sur-apprentissage et régularisation

Le **sur-apprentissage (overfitting)** se produit lorsque le modèle apprend trop bien les données d'entraînement, y compris le bruit et les particularités spécifiques à cet ensemble, ce qui le rend moins performant sur de nouvelles données non vues. La **régularisation** est un ensemble de techniques utilisées pour réduire le sur-apprentissage et améliorer la généralisation du modèle.

Early stopping, Dropout, régularisation L1/L2 (formulations mathématiques)

- **Early Stopping :** Cette technique consiste à arrêter l'entraînement lorsque la performance du modèle sur un ensemble de validation (données non utilisées pour l'entraînement) commence à se détériorer, même si la performance sur l'ensemble d'entraînement continue de s'améliorer. Cela permet d'éviter que le modèle n'apprenne trop les spécificités de l'ensemble d'entraînement.
- **Dropout :** Introduit par Hinton et al., le dropout est une technique de régularisation puissante où, pendant l'entraînement, un pourcentage aléatoire de neurones est désactivé (mis à zéro) à chaque itération. Cela empêche les neurones de co-dépendre excessivement et force le réseau à apprendre des représentations plus robustes.
- **Régularisation L1 (Lasso) et L2 (Ridge) :** Ces techniques ajoutent un terme de pénalité à la fonction de coût, ce qui encourage les poids du modèle à prendre des valeurs plus petites. Cela réduit la complexité du modèle et aide à prévenir le sur-apprentissage.
 - **Régularisation L1 :** Ajoute la somme des valeurs absolues des poids à la fonction de coût.

$$J_{L1}(\mathbf{w}, b) = J(\mathbf{w}, b) + \lambda \sum_j |w_j|$$

La régularisation L1 a tendance à rendre certains poids exactement égaux à zéro, ce qui peut être utile pour la sélection de caractéristiques.

- **Régularisation L2** : Ajoute la somme des carrés des poids à la fonction de coût.

$$J_{L2}(\mathbf{w}, b) = J(\mathbf{w}, b) + \lambda \sum_j w_j^2$$

La régularisation L2 (également appelée *weight decay*) encourage les poids à être petits mais rarement nuls. Elle est très couramment utilisée.

Où λ est le paramètre de régularisation, qui contrôle l'intensité de la pénalité.

Exemples concrets détaillés

Exemple 1 : classement de fleurs (Iris) avec un petit réseau – calcul manuel d'une itération

Considérons le célèbre jeu de données Iris, qui contient des mesures de quatre caractéristiques (longueur et largeur des sépales, longueur et largeur des pétales) pour trois espèces de fleurs Iris. Nous allons simplifier cela pour un problème de classification binaire (par exemple, distinguer Setosa des autres) avec un petit réseau.

Architecture :

- Couche d'entrée : 4 neurones (pour les 4 caractéristiques)
- Couche cachée : 2 neurones (avec fonction d'activation sigmoïde)
- Couche de sortie : 1 neurone (avec fonction d'activation sigmoïde pour la classification binaire)

Données d'entrée (simplifiées pour un seul exemple) :

- $x = [5.1, 3.5, 1.4, 0.2]$ (caractéristiques d'une fleur Iris)
- $y = 1$ (si c'est une Setosa, 0 sinon)

Initialisation des poids et biais (aléatoire) :

- $W^{[1]} = \begin{pmatrix} 0.1 & 0.2 & 0.3 & 0.4 \\ 0.5 & 0.6 & 0.7 & 0.8 \end{pmatrix}, b^{[1]} = \begin{pmatrix} 0.1 \\ 0.2 \end{pmatrix}$
- $W^{[2]} = (0.9 \quad 1.0), b^{[2]} = (0.3)$

Calcul manuel d'une itération (Forward Pass) :

1. Couche cachée :

- $z_1^{[1]} = (0.1 \cdot 5.1) + (0.2 \cdot 3.5) + (0.3 \cdot 1.4) + (0.4 \cdot 0.2) + 0.1 = 0.51 + 0.7 + 0.42 + 0.08 + 0.1 = 1.81$

- $z_2^{[1]} = (0.5 \cdot 5.1) + (0.6 \cdot 3.5) + (0.7 \cdot 1.4) + (0.8 \cdot 0.2) + 0.2 = 2.55 + 2.1 + 0.98 + 0.16 + 0.2 = 5.99$
- $a_1^{[1]} = \sigma(1.81) \approx 0.859$
- $a_2^{[1]} = \sigma(5.99) \approx 0.997$
- Donc, $a^{[1]} = \begin{pmatrix} 0.859 \\ 0.997 \end{pmatrix}$

2. Couche de sortie :

- $z^{[2]} = (0.9 \cdot 0.859) + (1.0 \cdot 0.997) + 0.3 = 0.7731 + 0.997 + 0.3 = 2.0701$
- $\hat{y} = \sigma(2.0701) \approx 0.888$

Calcul de la fonction de coût (Erreur quadratique) :

- $J = \frac{1}{2}(0.888 - 1)^2 = \frac{1}{2}(-0.112)^2 = \frac{1}{2} \cdot 0.012544 = 0.006272$

Calcul manuel d'une itération (Backward Pass - simplifié pour l'exemple) :

1. Gradient de la couche de sortie :

- $\frac{\partial J}{\partial \hat{y}} = \hat{y} - y = 0.888 - 1 = -0.112$
- $\sigma'(z^{[2]}) = \sigma(z^{[2]})(1 - \sigma(z^{[2]})) = 0.888(1 - 0.888) = 0.888 \cdot 0.112 \approx 0.099456$
- $\frac{\partial J}{\partial z^{[2]}} = -0.112 \cdot 0.099456 \approx -0.01114$
- $\frac{\partial J}{\partial W_1^{[2]}} = \frac{\partial J}{\partial z^{[2]}} \cdot a_1^{[1]} = -0.01114 \cdot 0.859 \approx -0.00957$
- $\frac{\partial J}{\partial W_2^{[2]}} = \frac{\partial J}{\partial z^{[2]}} \cdot a_2^{[1]} = -0.01114 \cdot 0.997 \approx -0.01111$
- $\frac{\partial J}{\partial b^{[2]}} = \frac{\partial J}{\partial z^{[2]}} = -0.01114$

2. Mise à jour des poids et biais (avec un taux d'apprentissage $\alpha = 0.1$) :

- $W_1^{[2]} \leftarrow 0.9 - 0.1 \cdot (-0.00957) = 0.9 + 0.000957 = 0.900957$
- $W_2^{[2]} \leftarrow 1.0 - 0.1 \cdot (-0.01111) = 1.0 + 0.001111 = 1.001111$
- $b^{[2]} \leftarrow 0.3 - 0.1 \cdot (-0.01114) = 0.3 + 0.001114 = 0.301114$

Ce processus se répète pour toutes les couches et tous les exemples d'entraînement, ajustant progressivement les poids et les biais pour minimiser la fonction de coût.

Exemple 2 : régression – prédiction d'un prix (formulation matricielle)

Pour un problème de régression, comme la prédiction du prix d'une maison en fonction de ses caractéristiques (taille, nombre de chambres, emplacement, etc.), nous utiliserions une fonction de coût comme l'erreur quadratique moyenne et une couche de sortie avec une activation linéaire (pas de fonction d'activation).

Formulation matricielle :

Soit X la matrice des caractéristiques d'entrée (chaque ligne est un exemple, chaque colonne est une caractéristique) et Y le vecteur des vrais prix.

- **Forward Pass :**

- $Z^{[1]} = XW^{[1]} + B^{[1]}$
- $A^{[1]} = \text{ReLU}(Z^{[1]})$ (ou toute autre fonction d'activation)
- $Z^{[2]} = A^{[1]}W^{[2]} + B^{[2]}$
- $\hat{Y} = Z^{[2]}$ (activation linéaire pour la régression)

- **Fonction de coût (MSE) :**

- $J = \frac{1}{2m} \|\hat{Y} - Y\|_F^2$ (où F est la norme de Frobenius)

- **Backward Pass (gradients pour la dernière couche) :**

- $\frac{\partial J}{\partial Z^{[2]}} = \frac{1}{m}(\hat{Y} - Y)$
- $\frac{\partial J}{\partial W^{[2]}} = (A^{[1]})^T \frac{\partial J}{\partial Z^{[2]}}$
- $\frac{\partial J}{\partial B^{[2]}} = \sum_{i=1}^m \frac{\partial J}{\partial Z_i^{[2]}}$

Les gradients pour les couches précédentes seraient calculés de manière similaire en utilisant la règle de la chaîne.

Aller plus loin (niveau expert)

Batch normalisation (formule + raison)

La **Batch Normalization** (BN) est une technique qui normalise les activations des couches intermédiaires dans un réseau de neurones. Elle a été introduite pour résoudre le problème du *Internal Covariate Shift*, qui se produit lorsque la distribution des entrées de chaque couche change pendant l'entraînement, rendant l'apprentissage plus difficile et plus lent. La BN normalise les activations pour qu'elles aient une moyenne de zéro et une variance de un, ce qui stabilise l'entraînement et permet d'utiliser des taux d'apprentissage plus élevés.

Formule :

Pour une mini-batch donnée $B = \{x_1, \dots, x_m\}$:

1. **Calcul de la moyenne et de la variance de la mini-batch :**

- $\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$
- $\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$

2. **Normalisation :**

- $\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$ (où ϵ est une petite constante pour éviter la division par zéro)

3. Mise à l'échelle et décalage (apprentissage de paramètres) :

$$\circ y_i = \gamma \hat{x}_i + \beta$$

Où γ et β sont des paramètres appris par le réseau, qui permettent au modèle de restaurer la capacité de représenter l'identité si nécessaire. La Batch Normalization est généralement appliquée avant la fonction d'activation.

Optimiseurs : Momentum, Adam (mise à jour des poids avec équations)

Les optimiseurs sont des algorithmes qui ajustent les poids et les biais du réseau pour minimiser la fonction de coût. La Descente de Gradient Stochastique (SGD) est l'optimiseur de base, mais des optimiseurs plus avancés peuvent accélérer la convergence et améliorer la performance.

- **Momentum** : Cet optimiseur aide à accélérer la SGD dans la direction pertinente et amortit les oscillations. Il ajoute un terme de «momentum» aux mises à jour des poids, qui est une fraction de la mise à jour précédente.

$$\circ v_t = \beta v_{t-1} + (1 - \beta) \frac{\partial J}{\partial \mathbf{w}}$$

$$\circ \mathbf{w} \leftarrow \mathbf{w} - \alpha v_t$$

Où v_t est la vitesse à l'itération t , et β est le coefficient de momentum (généralement 0.9).

- **Adam (Adaptive Moment Estimation)** : C'est l'un des optimiseurs les plus populaires et les plus efficaces. Il combine les avantages du Momentum et de RMSprop (un autre optimiseur adaptatif). Adam calcule des taux d'apprentissage adaptatifs pour chaque paramètre en se basant sur les estimations des premiers et deuxièmes moments des gradients.

- Initialisation : $m_0 = 0$ (premier moment estimé), $v_0 = 0$ (deuxième moment estimé)

- Pour chaque itération t :

- Calcul du gradient : $g_t = \frac{\partial J}{\partial \mathbf{w}_t}$

- Mise à jour des moments :

- $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$

- $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$

- Correction du biais des moments :

- $\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$

- $\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$

- Mise à jour des poids : $\mathbf{w}_t \leftarrow \mathbf{w}_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$

Où β_1 et β_2 sont les coefficients d'atténuation (généralement 0.9 et 0.999), et ϵ est une petite constante pour éviter la division par zéro.

Fonctions de coût personnalisées

Dans certains cas, les fonctions de coût standard (MSE, entropie croisée) peuvent ne pas être optimales pour un problème donné. Il est alors possible de définir des fonctions de coût personnalisées qui reflètent mieux les objectifs spécifiques de la tâche. Par exemple, dans les problèmes de détection d'objets, des fonctions de coût comme la *Intersection over Union (IoU)* peuvent être intégrées pour mieux pénaliser les erreurs de localisation des objets. La création de fonctions de coût personnalisées nécessite une bonne compréhension du problème et de la manière dont les erreurs doivent être pondérées.

Références

- [1] McCulloch, W. S., & Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *The bulletin of mathematical biophysics*, 5(4), 115-133. [Lien](#)
- [2] Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological review*, 65(6), 386. [Lien](#)
- [3] Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics* (pp. 249-256). JMLR Workshop and Conference Proceedings. [Lien](#)
- [4] He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision* (pp. 1026-1034). [Lien](#)
- [5] Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning* (pp. 448-456). PMLR. [Lien](#)
- [6] Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*. [Lien](#)